

Passing data to View

Passing Data to View from Controller

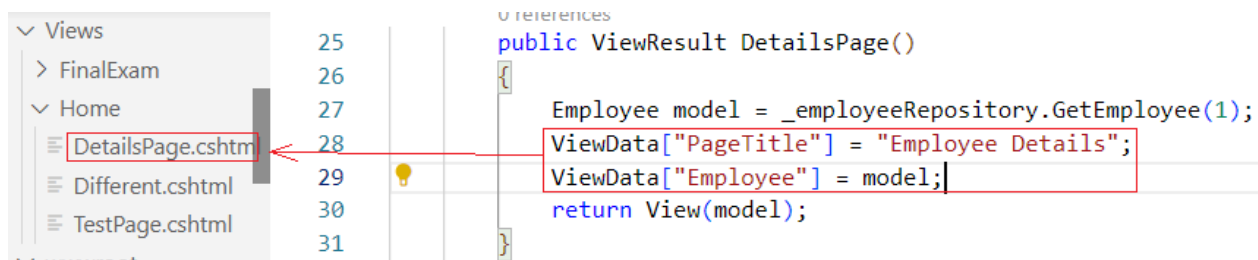
Different ways of passing data to a View from a Controller

- ViewData
 - ViewBag
 - Strongly Typed View
- Loosely Typed View

ViewData

- Dictionary of weakly typed objects
- Use string keys to store and retrieve data
- Dynamically resolved at runtime
- No compile - time type checking and Intellisens

използваме ViewData Dictionary - контейнери за данни с KeyWords, за да ги използваме за предаване другаде.



```
ViewData["PageTitle"] = "Employee Details";
```

```
ViewData["Employee"] = model;
```

```
Exam.cs | HomeController.cs | DetailsPage.cshtml X | Exams.cshtml |
eeManagement > Views > Home > DetailsPage.cshtml
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1>Hello!</h1>    @ - начало на C# код
  <h3>@ViewData["PageTitle"]</h3>    пренос на динамични данни
</body>
</html>
```

Дописваме: `<h3>@ViewData["PageTitle"]</h3>`

И като резултат прихвана динамично данните от контролера.



Hello!

Employee Details

Ако допуснем печатна грешка, няма да имаме никакъв проблем с кода, а просто няма да се изобрази заглавието на страницата:

```
<h1>Hello!</h1>    грешка
<h3>@ViewData["PageTitl_"]</h3>
```



Hello!

PageTitl_

В случая на PageTitle се използва string тип данни, затова не е нужно изречно да бъде обяснен какъв тип данни е, това е автоматично направено, но ако се използва какъвто и да е друг тип данни, той трябва обезателно да бъде упоменат. Такъв е

```
@{  
    var employee = ViewData["Employee"] as EmployeeManagement.Models.Employee;  
}
```

като сме посочили къде се намира този сложен custom type object class

```
<body>  
    <h1>Hello!</h1>  
    <h3>@ViewData["PageTitle"]</h3>  
    @{  
        сложен тип  
        var employee = ViewData["Employee"] as EmployeeManagement.Models.Employee;  
    }  
    class  
</body>
```

откъде .CS

```
meController.cs Employee.cs X DetailsPa  
agement > Models > Employee.cs > ...  
public class Employee  
{  
    3 references  
    public int Id { get; set; }  
  
    3 references  
    public string Name { get; set; }  
  
    2 references  
    public string Family { get; set; }  
  
    2 references  
    public string Phone { get; set; }  
  
    3 references  
    public string Email { get; set; }  
  
    2 references  
    public string Department { get; set; }  
}
```

Вече можем да изобразим тези данни, като ги вземем от класа:

Id, Name, Family, Phone, Email, Department и даже можем да ги подредим в таблица

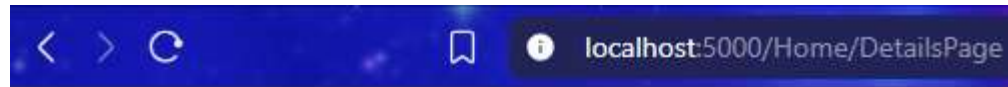
И ги подредим в таблица:

```
<table>
  <tr> table row
    <th>Id</th> table heading
    <th>Name</th>
    <th>Family</th>
    <th>Phone</th>
    <th>Email</th>
    <th>Department</th>
  </tr>
  <tr>
    <td>@employee.Id</td> table data
    <td>@employee.Name</td>
    <td>@employee.Family</td>
    <td>@employee.Phone</td>
    <td>@employee.Email</td>
    <td>@employee.Department</td>
  </tr>
</table>
```

```
<table style="border-style: solid;">
  <tr>
    <th>Id</th>
    <th>Name</th>
    <th>Family</th>
    <th>Phone</th>
    <th>Email</th>
    <th>Department</th>
  </tr>
  <tr>
    <td>@employee.Id</td>
    <td>@employee.Name</td>
    <td>@employee.Family</td>
    <td>@employee.Phone</td>
    <td>@employee.Email</td>
    <td>@employee.Department</td>
  </tr>
</table>
```

Таблицата може да се украси със CSS с border и т.н.

```
<table style="border-style: solid">
```



Hello!

Employee Details

Id	Name	Family	Phone	Email	Department
1	Мария	Георгиева	0123456789	Maria@omg.bg	Biology

Като изобразява името Мария, защото нейното id е прехвърлено за подаване в контролера:

```
public ActionResult DetailsPage()
{
    Employee model = _employeeRepository.GetEmployee(1);
    ViewData["Employee"] = model;
    ViewData["PageTitle"] = "Employee Details";
    return View(model);
}
```

Мария

Друг начин на представяне е:

```
<div>Id: @employee.Id</div>
<div>Name: @employee.Name</div>
<div>Family: @employee.Family</div>
<div>Phone: @employee.Phone</div>
<div>Email: @employee.Email</div>
<div>Department: @employee.D</div>
</body>
</html>
```

IntelliSense

abc Department

abc DOCTYPE

Т.е., за да прехвърляме данни динамично чрез С# първо трябва да извикаме модела, а после да изобразим данните където и както ни харесва – било чрез таблица или друг начин.

```

<body>
  <h1>Hello!</h1>
  <h3>@ViewData["PageTitle"]</h3>
  @{
    |   var employee = ViewData["Employee"] as EmployeeManagement.Models.Employee;
    |
  }

  <div>Id: @employee.Id</div>
  <div>Name: @employee.Name</div>
  <div>Family: @employee.Family</div>
  <div>Phone: @employee.Phone</div>
  <div>Email: @employee.Email</div>
  <div>Department: @employee.Department</div>

```



Hello!

Employee Details

Id: 1

Name: Мария

Family: Георгиева

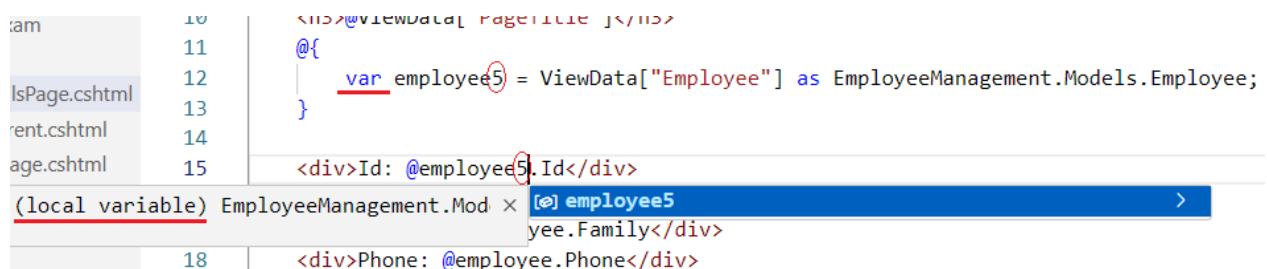
Phone: 0123456789

Email: Maria@omg.bg

Department: Biology

Проблемът е, че ViewData са лесни за печатни грешки (typographical errors) с този loosely type без Intellisense. Ако сгрешим и оставим Employee на едно място, Build-а е успешен, т.е. няма да разберем за грешката в CompileTime, а чак после в браузъра в Runtime и страницата ще крашне, т.е. съкращава времето без проверка за грешки Intellisense, не се компилира, а динамично се изпълнява по време на стартиране, затова принципно ViewData се избягва.

Например, спокойно можем да наречем променливата вместо employee да бъде employee5 и ако тя се извиква навсякъде, кодът пак ще работи:



Дори може вместо цялата дума, да опитаме само с буква a, b, c ...

```
<h3>@ViewData["PageTitle"]</h3>
@{
    var e = ViewData["Employee"] as EmployeeManagement.Models.Employee;
}
<div>Id: @e.Id</div>
<div>Name: @e.Name</div>
<div>Family: @e.Family</div>
<div>Phone: @e.Phone</div>
<div>Email: @e.Email</div>
<div>Department: @e.Department</div>
```

Кодът пак ще работи, защото това е просто локална променлива, на която ние избираме името и ѝ присвояваме стойността, която сме взели от ViewData на посочения клас.

Ако допуснем, обаче разминаване – грешка в изписването, няма да ни даде никакво съобщение с червено подчертаване или проблем при стартирането, а ще го видим чак в браузъра:

```
@{
    var e = ViewData["Employee1"] as EmployeeManagement.Models.Employee;
}
<div>Id: @e.Id</div>
<div>Name: @e.Name</div>
<div>Family: @e.Family</div>
<div>Phone: @e.Phone</div>
<div>Email: @e.Email</div>
<div>Department: @e.Department</div>
```



An unhandled exception occurred while processing the request.

NullReferenceException: Object reference not set to an instance of an object.

AspNetCoreGeneratedDocument.Views_Home_DetailsPage.<ExecuteAsync>b__8_1() in DetailsPage.cshtml, line 12

Stack

Query

Cookies

Headers

Routing

NullReferenceException: Object reference not set to an instance of an object.

AspNetCoreGeneratedDocument.Views_Home_DetailsPage.<ExecuteAsync>b__8_1() in DetailsPage.cshtml

12. var e = ViewData["Employee1"] as EmployeeManagement.Models.Employee;

Microsoft.AspNetCore.Razor.Runtime.TagHelpers.TagHelperExecutionContext.SetOutputContentAsync()

ViewData използва произволни random string keys, които при грешка с изписването се компилират, а тя се открива чак при динамично презареждане на страницата в runtime, като крашне. Затова ViewBag е по-добър.